

Informe Técnico Grupal

Requerimientos, Modelamiento y Metodología Ágil

Traduce el caso de Viajes Aventura (agencia de viajes que hoy opera con planilla de cálculo, cuaderno de reservas y mensajería) en requerimientos formales, en los diagramas BPMN, de casos de uso y de clases que modelan la solución, y en la metodología ágil, arquitectura y seguridad con que el equipo la construye.

Evaluación 4 · Rúbrica N°2: Informe Técnico Grupal y Defensa Argumentativa Individual (30%)

Asignatura: Programación Orientada a Objeto Seguro (TI3021) · INACAP Valparaíso

Estudiantes:

Logan Silva Jara (Líder de Proyecto) — Dominios Clientes y seguridad, y Reservas

Camilo Sepúlveda — Dominios Destinos y Paquetes

Profesor: Rubén Alejandro Schnettler Lucero

Fecha de entrega: 05 de octubre de 2026

Índice

1. Introducción y resumen del caso	3
2. Requerimientos funcionales y no funcionales	4
2.1 Requerimientos funcionales	4
2.2 Requerimientos no funcionales	5
2.3 Verificación de requerimientos contra la problemática	5
3. Modelamiento de la solución	6
3.1 Diagramas de procesos de negocio (BPMN)	8
3.2 Diagrama de casos de uso	12
3.3 Diagrama de clases UML	14
3.4 Trazabilidad entre requerimientos y modelos	16
4. Metodología de trabajo ágil	18
4.1 Roles del equipo	18
4.2 Product Backlog	18
4.3 Sprint Backlog y cronograma	18
5. Arquitectura y persistencia de datos	19
5.1 Arquitectura	19
5.2 Persistencia de datos	19
5.3 CRUD por dominio	20
6. Justificación técnica y uso crítico de herramientas de IA	22
7. Seguridad	24
7.1 Autenticación	24
7.2 Protección de datos sensibles	24
7.3 Evaluación de seguridad asistida por IA	24
8. Conclusiones	27
En síntesis, el sistema es funcional, está probado y desplegado, y deja documentados sus límites y sus decisiones.	27
Enlaces del proyecto	27
Referencias bibliográficas	27

1. Introducción y resumen del caso

Viajes Aventura es una agencia de viajes de Valparaíso, creada hace dos años por tres socios (Paulina Ovalle, Matías Bórquez e Ignacio Salas), que arma y vende paquetes turísticos combinando destinos que ella misma ha recorrido. Hoy no cuenta con ningún sistema: el catálogo vive en una planilla de cálculo compartida, las reservas se anotan en un cuaderno y se confirman por mensajería, y los datos de los clientes quedan dispersos entre ambos medios.

Esta operación manual genera errores concretos y medibles en la última temporada: 19 reservas duplicadas, 6 aceptadas por sobre el cupo real de un paquete, 3 tomadas sobre paquetes con fecha de salida ya vencida, y 4 paquetes publicados con un precio distinto del efectivamente cobrado — porque el costo de un destino se corrigió después de la venta. Este informe traduce ese problema en requerimientos de sistema, modela la solución con notación UML y BPMN, define la metodología de trabajo del equipo, y documenta la arquitectura, la seguridad y el uso crítico de herramientas de IA aplicado durante el desarrollo.

El detalle completo del caso, las reglas de negocio confirmadas por los socios (R1 a R17) y el alcance acordado se documentan en el Readme.md del repositorio; este informe se concentra en los entregables exigidos por la rúbrica de la Unidad 4.

2. Requerimientos funcionales y no funcionales

Los requerimientos de esta sección se levantaron directamente desde las 17 reglas de negocio confirmadas por los socios de Viajes Aventura (R1 a R17) y desde el alcance acordado del proyecto, sin reutilizar un levantamiento anterior. Se clasifican en funcionales (FR) y no funcionales (RNF), y se priorizan con un criterio único: una prioridad Alta corresponde a un requerimiento que impide una falla real y medible de la temporada pasada (duplicados, sobreventa de cupo, fechas vencidas, precios inconsistentes, exposición de datos sensibles) o que es condición de seguridad básica; Media corresponde a una mejora de calidad que no bloquea la operación central del negocio.

2.1 Requerimientos funcionales

ID	Requerimiento	Prioridad	Origen
FR-01	Registrar un destino con nombre único en el catálogo, zona, descripción, duración en días y costo base por persona mayor a cero.	Alta	R1, R2
FR-02	Modificar los datos de un destino existente.	Alta	R1
FR-03	Dar de baja un destino: eliminarlo del catálogo si no pertenece a ningún paquete; si pertenece a alguno, marcarlo no disponible en vez de eliminarlo.	Alta	R8
FR-04	Listar los destinos del catálogo, distinguiendo los disponibles de los marcados no disponibles.	Alta	R1, R8
FR-05	Crear un paquete combinando entre 2 y 5 destinos existentes sin repetir ninguno, con nombre, fecha de salida, fecha de regreso posterior a la salida y cupo máximo mayor a cero.	Alta	R3, R5
FR-06	Calcular el precio por persona del paquete como la suma de los costos base de los destinos incluidos, más el margen de operación definido por el administrador (margen mayor o igual a cero).	Alta	R6
FR-07	Publicar un paquete, fijando su precio definitivo: si después cambia el costo de un destino, los paquetes ya publicados conservan el precio con que se publicaron.	Alta	R7
FR-08	Consultar el cupo disponible de un paquete (cupos máximo menos las reservas vigentes).	Alta	R14
FR-09	Registrar un cliente con nombre, RUT, correo electrónico único, teléfono y contraseña.	Alta	R9
FR-10	Autenticar a un cliente mediante su correo y contraseña.	Alta	R9, R10
FR-11	Restringir reservar y consultar reservas a clientes autenticados; cada cliente accede únicamente a sus propias reservas.	Alta	R11
FR-12	Registrar una reserva de un cliente autenticado sobre un paquete publicado, indicando la cantidad de personas (mínimo uno), calculando el total como el precio del paquete por la cantidad de personas, y registrando la fecha de emisión.	Alta	R12, R13, R16
FR-13	Rechazar una reserva cuando la cantidad de personas solicitada supera el cupo disponible del paquete.	Alta	R14

ID	Requerimiento	Prioridad	Origen
FR-14	Rechazar una reserva sobre un paquete cuya fecha de salida ya pasó.	Alta	R15
FR-15	Permitir a un cliente autenticado consultar el historial completo de sus propias reservas.	Media	R11, R12
FR-16	Cancelar una reserva propia hasta la fecha de salida del paquete; libera su cupo.	Alta	S4, R11, R14
FR-17	Modificar la cantidad de personas de una reserva propia: la reserva anterior queda cancelada y se crea una nueva al precio vigente.	Alta	S4, R13, R14

2.2 Requerimientos no funcionales

ID	Tipo	Requerimiento	Prioridad	Origen
RNF-01	Seguridad	Las contraseñas de los clientes se almacenan únicamente como hash, nunca en texto plano.	Alta	R10
RNF-02	Seguridad	El RUT y el teléfono de un cliente no se muestran en listados ni se incluyen en mensajes de error.	Alta	R17
RNF-03	Seguridad	Toda operación de reservar o consultar reservas verifica la autenticación del lado del servidor, no solo en la interfaz.	Alta	R11
RNF-04	Persistencia	Los destinos, paquetes, clientes y reservas se almacenan en una base de datos que conserva la información entre reinicios del sistema, reemplazando la planilla y el cuaderno actuales.	Alta	Alcance
RNF-05	Integridad	Las validaciones de negocio sobre cupo, fechas y montos se ejecutan en el servidor; el sistema no se apoya en validaciones hechas solo del lado del cliente.	Alta	R14, R15
RNF-06	Usabilidad	El sistema se utiliza desde un navegador web, sin requerir instalación adicional para el cliente final.	Media	Alcance

2.3 Verificación de requerimientos contra la problemática

Cada fila de este cuadro conecta una cifra o un hecho documentado en la sección 1 con el o los requerimientos de 2.1/2.2 que lo resuelven. Ningún requerimiento de esta sección se incorporó sin que respondiera a una regla de negocio, una falla documentada o un límite de alcance explícito:

Falla u origen documentado	Requerimientos que la resuelven
19 reservas duplicadas detectadas al cierre de temporada.	FR-11 (identificación del cliente mediante autenticación), FR-12 (cada reserva queda registrada una sola vez, con su propia fecha de emisión).
6 reservas aceptadas por sobre el cupo real del paquete.	FR-08 (consultar cupo disponible), FR-13 (rechazar la reserva que exceda el cupo).
3 reservas tomadas sobre paquetes con fecha de salida ya vencida.	FR-14 (rechazar reserva sobre paquete vencido).

Falla u origen documentado	Requerimientos que la resuelven
4 paquetes publicados con un precio distinto del efectivamente cobrado.	FR-06 (precio calculado por el sistema, no a mano), FR-07 (precio fijado al publicar, inmutable ante cambios posteriores).
31 consultas de clientes resueltas revisando el cuaderno hoja por hoja.	FR-15 (historial de reservas auto consultable por el propio cliente).
4 destinos que ya no se operan, pero siguen apareciendo en el catálogo.	FR-03 (baja lógica del destino), FR-04 (listar solo los disponibles).
RUT y teléfono de clientes dispersos en planilla, cuaderno y mensajería, expuestos a cualquiera que abra esos archivos.	RNF-01 (hash de contraseña), RNF-02 (RUT/teléfono ocultos en listados y errores).

No queda ninguna cifra de la sección 1 sin un requerimiento que la aborde, ni un requerimiento de 2.1 o 2.2 sin una regla de negocio, falla documentada o límite de alcance que lo origine.

3. Modelamiento de la solución

Esta sección traduce los requerimientos de la sección 2 en modelos de la solución, en tres niveles complementarios: los procesos de negocio (BPMN, 3.1), que muestran paso a paso cómo interactúan el Administrador, el Cliente y el sistema; los casos de uso (3.2), que resumen qué puede hacer cada actor; y el diagrama de clases UML (3.3), que traduce ambos en la estructura orientada a objetos que se implementó en Python. La sección 3.4 cierra con una matriz que verifica, requerimiento por requerimiento, que cada FR y RNF quedó representado en los modelos. Todos los diagramas se construyeron desde cero a partir de FR-01 a FR-15 y RNF-01 a RNF-06; sus fuentes editables están en la carpeta docs/diagramas/ del repositorio.

Supuestos de modelamiento

El caso advierte que no entrega toda la información («Los vacíos del caso») y pide declarar y fundamentar los supuestos. Antes de modelar se fijaron los siguientes; cada uno queda reflejado en al menos un diagrama:

ID	Supuesto	Fundamento técnico
S1	El Administrador es cualquiera de los tres socios. Se autentica igual que un cliente (correo y contraseña) y es el único que modifica destinos y paquetes.	Responde al vacío «quién puede modificar el catálogo». Restringir la edición a un rol autenticado elimina la planilla editada libremente por los tres socios, origen de los destinos duplicados.
S2	Un paquete nace en estado BORRADOR y solo se muestra y acepta reservas cuando está PUBLICADO.	R7 fija el precio al publicar: sin un estado explícito no se distingue un precio provisional de uno definitivo.
S3	Cuando la fecha de salida pasa, el paquete no se elimina: sigue en la base de datos, pero deja de aceptar reservas.	Responde al vacío «cómo se comporta un paquete cuando su temporada termina». Las reservas pasadas lo referencian y el historial del cliente (FR-15) debe seguir mostrándolas.
S4	Las reservas pueden cancelarse hasta la fecha de salida, liberando su cupo, y modificarse en cantidad de personas: la modificación cancela la reserva original y crea una nueva al precio vigente (R13).	Responde al vacío sobre el cliente que desiste. El alcance acordado no la incluye y la devolución del dinero depende de transferencias, que están fuera de alcance. Si se agrega, el cupo se recalcula solo (S5).
S5	El cupo disponible no se almacena: se calcula como cupo máximo menos las personas ya reservadas.	Un contador guardado aparte puede desincronizarse, que es lo que produjo las 6 reservas sobre el cupo. Calcularlo siempre desde las reservas lo hace consistente por construcción (R14).

ID	Supuesto	Fundamento técnico
S6	Los montos se manejan en pesos chilenos enteros y el margen como fracción (0,20 = 20 %).	Los costos del caso son enteros en CLP; usar enteros evita errores de redondeo en el total cobrado (R13).

3.1 Diagramas de procesos de negocio (BPMN)

Los tres diagramas usan notación BPMN 2.0: un pool «Viajes Aventura» dividido en carriles (lanes) por participante —Administrador o Cliente, y Sistema—; un evento de inicio (círculo de borde simple); eventos de fin (círculo de borde grueso), uno por cada resultado distinto para el negocio; tareas (rectángulos redondeados); compuertas exclusivas XOR (rombo con X) para cada decisión; y el almacén de datos «Base de datos», unido con una asociación punteada a la tarea que persiste información. Cada tarea indica entre corchetes el requerimiento que implementa, de modo que la trazabilidad se lee directamente sobre el diagrama. Todas las validaciones de negocio ocurren en el carril Sistema y no en el del actor, lo que materializa RNF-05.

Diagrama 1 — Gestión del catálogo de destinos (Administrador), cubre FR-01 a FR-04

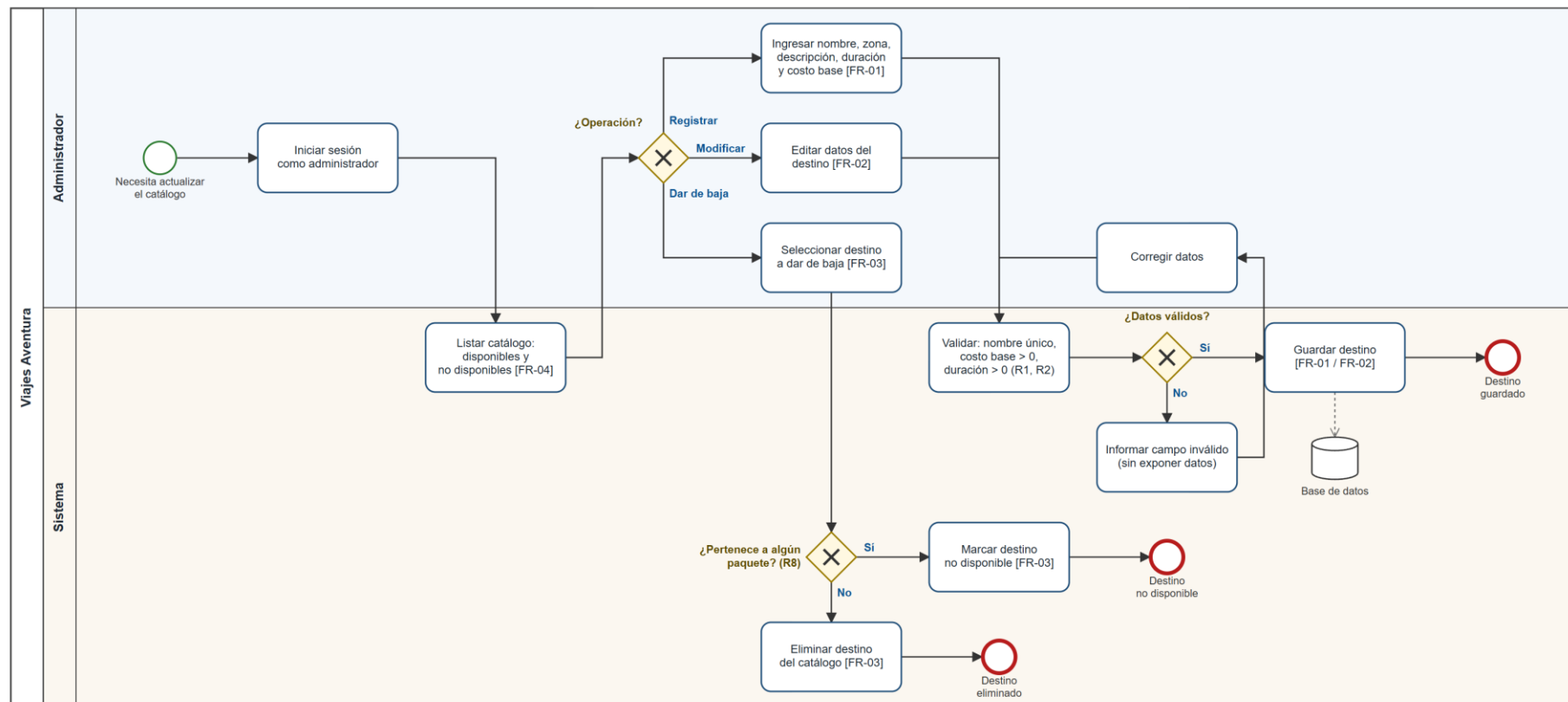


Figura 1 · Proceso BPMN de gestión del catálogo de destinos

Diagrama 2 — Armado y publicación de un paquete (Administrador), cubre FR-05 a FR-07

El Administrador define nombre, fechas, cupo y margen, y elige entre 2 y 5 destinos de una lista que el sistema filtra para ofrecer solo los disponibles (R8). El sistema valida R3, R5 y R6 y, si alguna regla falla, devuelve al Administrador a corregir. Con datos válidos es el sistema —no la persona— el que calcula el precio por persona (FR-06), lo que elimina el error de digitación que publicó un paquete a la décima parte de su valor. El Administrador decide publicar o dejar el paquete en borrador (S2); al publicar, el precio queda fijado (FR-07) y deja de depender de cambios posteriores en el costo de los destinos.

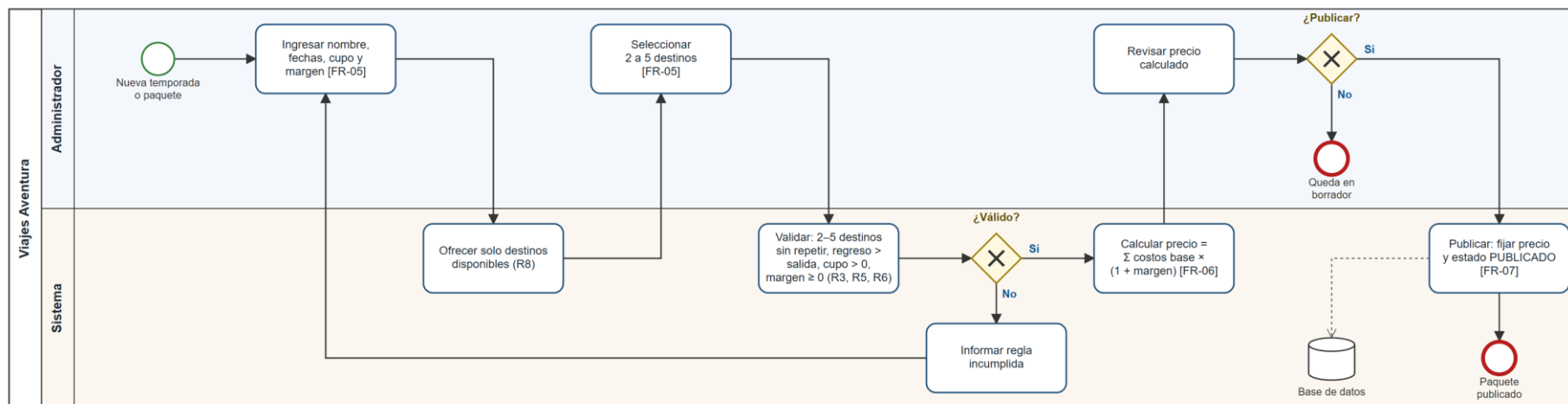


Figura 2 · Proceso BPMN de armado y publicación de un paquete

Diagrama 3 — Registro, autenticación y reserva (Cliente), cubre FR-08 a FR-15

El Cliente consulta los paquetes publicados con su cupo disponible (FR-08). Para reservar, la compuerta «¿Autenticado?» obliga a iniciar sesión o a registrarse primero (R11, FR-11); el registro guarda la contraseña solo como hash (RNF-01) y un fallo de credenciales devuelve un mensaje genérico que no revela qué dato falló (RNF-02). Las dos compuertas siguientes reflejan la pista del caso: la fecha vencida (FR-14) se compara con la fecha del día y el exceso de cupo (FR-13) con un dato que lleva el sistema, por eso son decisiones separadas con rechazos distintos. Si ambas se cumplen, el sistema registra la reserva con su total y su fecha de emisión (FR-12), y el Cliente puede verla en su historial (FR-15).

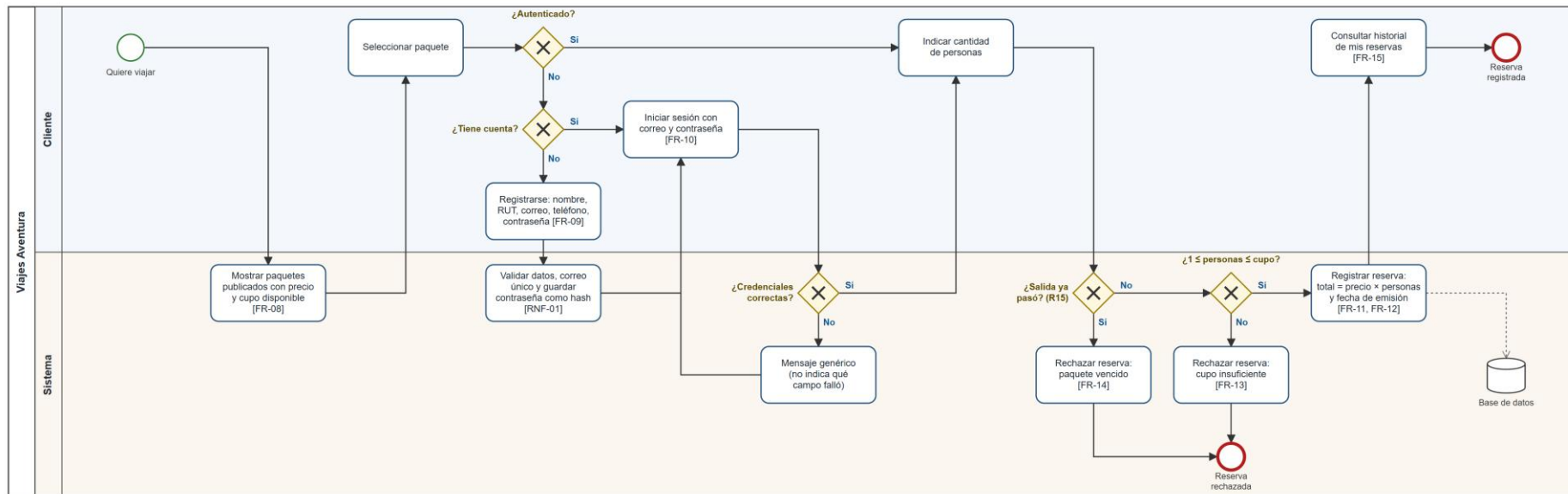


Figura 3 · Proceso BPMN de registro, autenticación y reserva de un paquete

3.2 Diagrama de casos de uso

El diagrama resume las interacciones de los dos actores del caso con el sistema. «Iniciar sesión» es el único caso de uso compartido (S1). Las relaciones «include» expresan comportamiento obligatorio: «Reservar paquete» y «Consultar historial» incluyen «Iniciar sesión» porque R11 no permite reservar ni consultar sin autenticación; «Reservar paquete» incluye «Verificar cupo y fecha de salida» porque ninguna reserva puede saltarse FR-13 y FR-14; y «Crear paquete» incluye «Calcular precio del paquete» porque el precio nunca se ingresa a mano (FR-06). No se usó «extend» porque, dentro del alcance, ningún comportamiento es opcional respecto de un caso base.

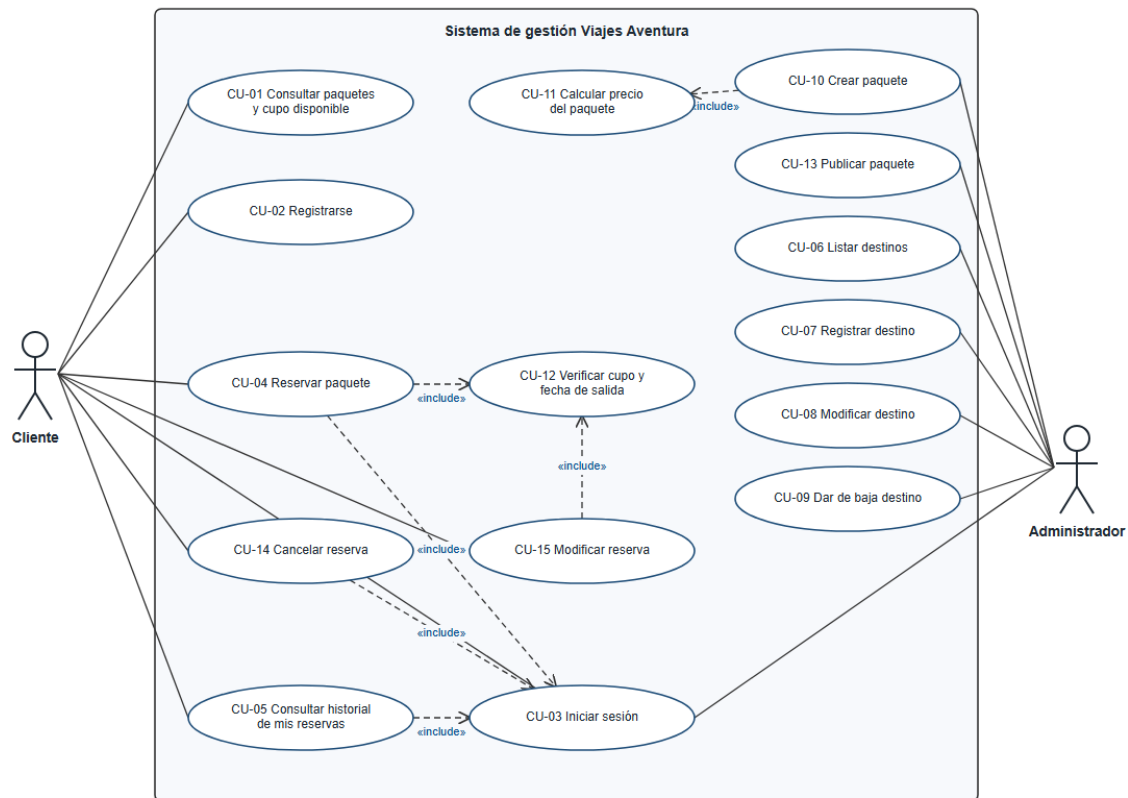


Figura 4 · Diagrama de casos de uso, actores Cliente y Administrador

Detalle de cada caso de uso:

Caso de uso	Actor	Descripción	Requerimientos
CU-01 Consultar paquetes y cupo disponible	Cliente	Ver los paquetes publicados y vigentes con su precio por persona y su cupo disponible.	FR-08
CU-02 Registrarse	Cliente	Crear una cuenta con nombre, RUT, correo único, teléfono y contraseña.	FR-09, RNF-01
CU-03 Iniciar sesión	Cliente, Administrador	Autenticarse con correo y contraseña; habilita los casos de uso restringidos.	FR-10, RNF-03
CU-04 Reservar paquete	Cliente	Reservar un paquete publicado indicando la cantidad de personas. Incluye CU-03 y CU-12.	FR-11, FR-12
CU-05 Consultar historial de mis reservas	Cliente	Ver solo las reservas propias, presentes y pasadas. Incluye CU-03.	FR-11, FR-15
CU-06 Listar destinos	Administrador	Ver el catálogo, distinguiendo destinos disponibles y no disponibles.	FR-04

Caso de uso	Actor	Descripción	Requerimientos
CU-07 Registrar destino	Administrador	Agregar un destino con nombre único y costo base mayor que cero.	FR-01
CU-08 Modificar destino	Administrador	Corregir los datos de un destino existente.	FR-02
CU-09 Dar de baja destino	Administrador	Eliminar el destino o marcarlo no disponible, según pertenezca o no a algún paquete.	FR-03
CU-10 Crear paquete	Administrador	Armar un paquete con 2 a 5 destinos, fechas, cupo y margen. Incluye CU-11.	FR-05
CU-11 Calcular precio del paquete	— (incluido)	Sumar los costos base de los destinos y aplicar el margen.	FR-06
CU-12 Verificar cupo y fecha de salida	— (incluido)	Rechazar la reserva si excede el cupo disponible o si la salida ya pasó.	FR-13, FR-14
CU-13 Publicar paquete	Administrador	Fijar el precio definitivo y dejar el paquete disponible para reservas.	FR-07
CU-14 Cancelar reserva	Cliente	Cancelar una reserva propia hasta la fecha de salida; el cupo vuelve a quedar disponible. Incluye CU-03.	FR-16
CU-15 Modificar reserva	Cliente	Cambiar la cantidad de personas: la reserva actual queda cancelada y se crea una nueva al precio vigente. Incluye CU-03 y CU-12.	FR-17

3.3 Diagrama de clases UML

El modelo se presenta en dos vistas: el dominio (Figura 5), con las entidades del negocio y sus reglas, y la capa de persistencia (Figura 6), que las almacena en SQLite. Ambas son la base directa de la implementación en Python del Paso 4 (criterio 4.1.4).

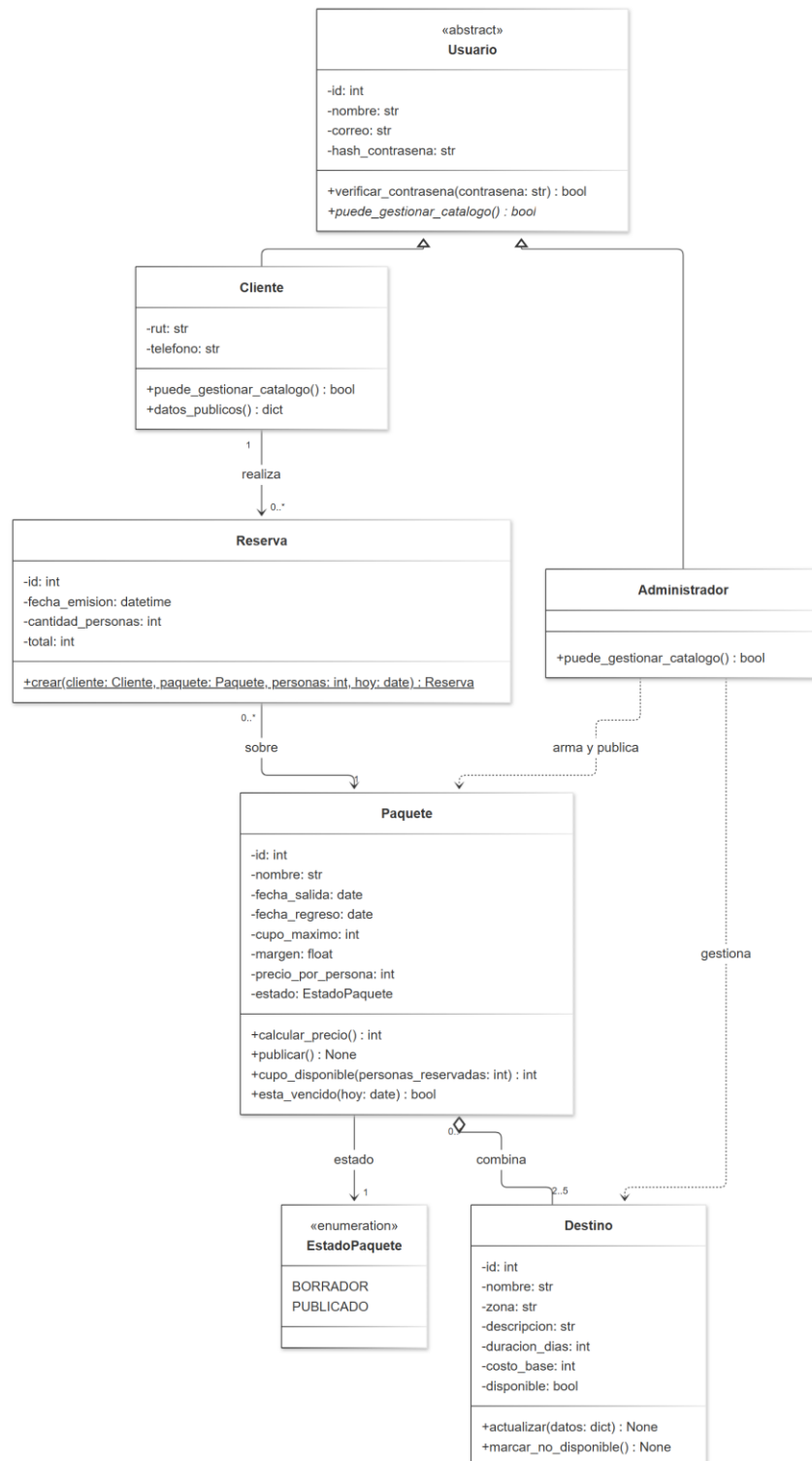


Figura 5 · Diagrama de clases UML: modelo de dominio

Decisiones de diseño orientado a objetos:

- **Encapsulamiento:** todos los atributos son privados (-) y el estado solo cambia mediante métodos que aplican las reglas. El precio no tiene un método de asignación libre: solo `calcular_precio()` y `publicar()` lo modifican (R6, R7), y disponible solo cambia con `marcar_no_disponible()` (R8).
- **Abstracción y herencia:** Usuario es una clase abstracta que reúne lo común a Cliente y Administrador (identidad, correo, hash de la contraseña y `verificar_contraseña()`), para no duplicar la lógica de autenticación (FR-10, RNF-01). Ninguna clase tiene un atributo de contraseña en texto plano.
- **Polimorfismo:** `puede_gestionar_catalogo()` es abstracto en Usuario y cada subclase lo redefine (Cliente devuelve falso; Administrador, verdadero). El servidor autoriza las operaciones sobre el catálogo llamando a ese método, sin preguntar por el tipo concreto del usuario (RNF-03, S1).
- **Relaciones y cardinalidades:** Paquete agrega 2..5 Destino (rombo vacío: el destino existe por sí mismo y puede estar en varios paquetes, R3 y R4); Cliente realiza 0..* Reserva y cada Reserva es sobre exactamente 1 Paquete (R12); Paquete tiene un EstadoPaquete, BORRADOR o PUBLICADO (S2).
- **Reglas dentro del dominio:** `Reserva.crear()` es un método de fábrica (estático, subrayado) que solo construye reservas válidas: exige al menos una persona (R16), cupo disponible (R14) y fecha de salida futura (R15), y calcula el total una sola vez (R13). Como el total no tiene método de asignación, no vuelve a cambiar. `Cliente.datos_publicos()` entrega los datos del cliente sin RUT ni teléfono para cualquier listado (R17, RNF-02).
- **Cupo calculado, no almacenado:** Paquete no guarda el cupo disponible; `cupo_disponible()` lo deriva del cupo máximo y de las personas ya reservadas que entrega el repositorio (S5).

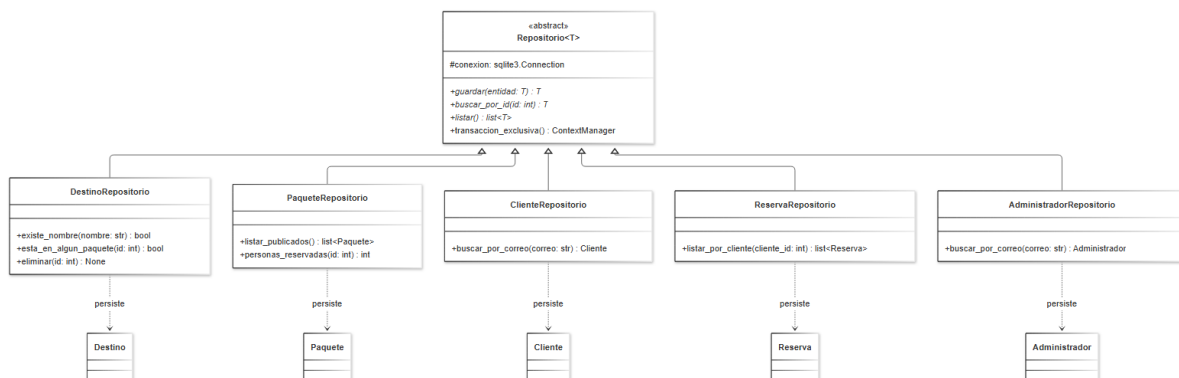


Figura 6 · Diagrama de clases UML: capa de persistencia

`Repositorio<T>` es una clase abstracta genérica que define el contrato común de persistencia (guardar, `buscar_por_id`, `listar`) sobre una conexión `sqlite3`. Cada subclase lo implementa para su entidad y agrega solo las consultas que su dominio necesita: `esta_en_algun_paquete()` para R8, `personas_reservadas()` para R14, `buscar_por_correo()` para FR-10 y `listar_por_cliente()` para FR-15. Así las clases de dominio no contienen SQL, las consultas quedan concentradas en un solo lugar para usar siempre parámetros (prevención de inyección SQL, sección 7) y se cumple RNF-04 sin acoplar las reglas de negocio a la base de datos.

3.4 Trazabilidad entre requerimientos y modelos

La matriz recorre los 21 requerimientos de la sección 2 y ubica, para cada uno, la tarea o compuerta BPMN, el caso de uso y la clase o método que lo representan:

Req.	BPMN (diagrama: elemento)	Caso de uso	Clase / método
FR-01	D1: Ingresar datos → Validar → Guardar destino	CU-07	Destino; DestinoRepositorio.existe_nombre()
FR-02	D1: Editar datos → Validar → Guardar destino	CU-08	Destino.actualizar()
FR-03	D1: compuerta «¿Pertenece a algún paquete?» → Marcar no disponible / Eliminar	CU-09	Destino.marcar_no_disponible(); DestinoRepositorio.esta_en_algun_p aquete(), eliminar()
FR-04	D1: Listar catálogo	CU-06	Destino.disponible; DestinoRepositorio.listar()
FR-05	D2: Ingresar datos, Seleccionar 2 a 5 destinos, Validar	CU-10	Paquete; agregación Paquete 0..* — 2..5 Destino
FR-06	D2: Calcular precio	CU-11 («include» de CU-10)	Paquete.calcular_precio()
FR-07	D2: compuerta «¿Publicar?» → Publicar	CU-13	Paquete.publicar(); EstadoPaquete
FR-08	D3: Mostrar paquetes con cupo disponible	CU-01	Paquete.cupo_disponible(); PaqueteRepositorio.personas_reserv adas()
FR-09	D3: Registrarse → Validar datos y guardar	CU-02	Cliente; ClienteRepositorio.guardar()
FR-10	D3: Iniciar sesión → compuerta «¿Credenciales correctas?»	CU-03	Usuario.verificar_contraseña(); ClienteRepositorio.buscar_por_corre o()
FR-11	D3: compuerta «¿Autenticado?» antes de reservar	«include» CU-04 y CU-05 → CU-03	Asociación Cliente 1 — 0..* Reserva
FR-12	D3: Registrar reserva	CU-04	Reserva.crear(); fecha_emision, cantidad_personas, total
FR-13	D3: compuerta «¿1 ≤ personas ≤ cupo?» → Rechazar	CU-12 («include» de CU-04)	Paquete.cupo_disponible(); Reserva.crear()
FR-14	D3: compuerta «¿Salida ya pasó?» → Rechazar	CU-12 («include» de CU-04)	Paquete.esta_vencido(hoy)
FR-15	D3: Consultar historial de mis reservas	CU-05	ReservaRepositorio.listar_por_client e()
FR-16	D3: Cancelar reserva → Validar vigencia → Marcar cancelada (libera cupo)	CU-14	Reserva.cancelar(); ReservaRepositorio.marcar_cancelad a()
FR-17	D3: Modificar reserva → Validar cupo sin la reserva propia → Cancelar anterior y crear nueva	CU-15	Reserva.reemplazar_por(); Paquete.cupo_disponible()
RNF-01	D3: guardar contraseña como hash	CU-02, CU-03	Usuario.hash_contraseña (sin contraseña en claro)
RNF-02	D3: mensaje genérico de error; D1: informar campo inválido sin exponer datos	CU-02, CU-03	Cliente.datos_publicos(); rut y telefono privados

Req.	BPMN (diagrama: elemento)	Caso de uso	Clase / método
RNF-03	D1 y D3: inicio de sesión verificado en el carril Sistema	CU-03 (incluido por CU-04 y CU-05)	Usuario.puede_gestionar_catalogo() (polimorfismo)
RNF-04	D1, D2 y D3: almacén de datos «Base de datos»	Todos	Repositorio<T> y sus cinco subclases
RNF-05	D1, D2 y D3: toda validación ocurre en el carril Sistema	CU-11, CU-12	calcular_precio(), esta_vencido(), cupo_disponible(), Reserva.crear()
RNF-06	— (no describe un proceso)	—	Se resuelve en la arquitectura (sección 5)

Validación de coherencia realizada sobre la matriz: (1) los 17 requerimientos funcionales aparecen en un diagrama BPMN, en un caso de uso y en una clase o método; (2) los 15 casos de uso y las 7 clases de dominio se originan en al menos un requerimiento o supuesto declarado, y no se modeló nada fuera del alcance (por ejemplo, no hay pagos ni facturación); (3) RNF-06 es el único requerimiento sin un elemento de modelo propio, porque describe el medio de acceso (un navegador web) y no un proceso ni una entidad, y se aborda en la arquitectura (sección 5).

4. Metodología de trabajo ágil

Esta sección se redactó de nuevo el 1 de octubre de 2026, descartando el cronograma anterior (que llegaba solo hasta el 23-30 de septiembre y ya no correspondía a la fecha real de reinicio del proyecto): roles, Product Backlog y Sprint Backlog se definen aquí a partir de los requerimientos vigentes (FR-01 a FR-15, RNF-01 a RNF-06) y del calendario real que queda hasta la entrega del 5 de octubre de 2026.

4.1 Roles del equipo

Logan Silva es el Líder de Proyecto, rol definido en la guía de evaluación: coordina las actividades del equipo, gestiona las reuniones de seguimiento, supervisa el cumplimiento de tareas y mantiene la comunicación con el docente. El equipo interpreta además el rol de Product Owner a partir del propio caso (las necesidades de los tres socios de Viajes Aventura), consolidado y priorizado por el Líder de Proyecto. La responsabilidad técnica se divide en dos franjas verticales por flujo de negocio: Camilo Sepúlveda cubre el lado de catálogo (Destinos alimenta a Paquetes), y Logan Silva cubre el lado de operación con el cliente (Clientes y seguridad alimenta a Reservas, que depende de la autenticación):

Integrante	Rol	Dominios técnicos	Reglas
Logan Silva	Líder de Proyecto + Developer	Clientes y seguridad, y Reservas	R9-R17
Camilo Sepúlveda	Developer	Destinos y Paquetes	R1-R8

4.2 Product Backlog

Historias de usuario derivadas de los requerimientos funcionales y no funcionales de la sección 2, priorizadas y asignadas por dominio. Las cinco cubren la totalidad de los requerimientos vigentes, sin dejar ninguno sin una historia que lo implemente:

ID	Historia de usuario	Requerimientos	Prioridad	Responsable
HU-01	Como administrador quiero gestionar el catálogo de destinos (registrar, modificar, dar de baja lógica, listar) para mantenerlo correcto.	FR-01 a FR-04	Alta	Camilo Sepúlveda
HU-02	Como administrador quiero armar y publicar paquetes con precio calculado automáticamente y cupo consultable.	FR-05 a FR-08	Alta	Camilo Sepúlveda
HU-03	Como visitante quiero registrarme e iniciar sesión de forma segura para poder reservar.	FR-09, FR-10, RNF-01 a RNF-03	Alta	Logan Silva
HU-04	Como cliente autenticado quiero reservar un paquete respetando cupo y fecha, y ver mi historial de reservas.	FR-11 a FR-15	Alta	Logan Silva
HU-05	Como equipo queremos que la información persista en una base de datos y que el sistema se entregue con un solo comando, para que la evaluación no dependa de instalar dependencias extra.	RNF-04 a RNF-06	Alta	Equipo completo

4.3 Sprint Backlog y cronograma

Con la entrega fijada para el 5 de octubre de 2026 y el reinicio del proyecto el 1 de octubre, el trabajo se organiza en tres iteraciones cortas más la entrega formal:

Sprint	Fechas	Objetivo	Entregable
Sprint 0	1 oct	Análisis del caso, levantamiento de requerimientos y modelado UML/BPMN.	Informe Técnico, secciones 1 a 3 completas (requerimientos, BPMN, casos de uso, clases).
Sprint 1	2-3 oct	Implementación en paralelo de los 4 dominios, con persistencia y autenticación.	CRUD funcional de Destinos, Paquetes, Clientes y Reservas; hash de contraseña y JWT operativos.
Sprint 2	4 oct	Integración de los dominios, pruebas de punta a punta, revisión de seguridad y cierre de las secciones 4 a 8 del informe.	Sistema integrado y probado; Informe Técnico completo.
Entrega	5 oct	Carga de la documentación y el código en el AAI; defensa argumentativa individual.	Informe Técnico Grupal + código fuente + defensa de cada integrante.

Los Sprints 0, 1 y 2 se completaron al 1 de octubre (ver Cambios 3 a 11 en VALIDACION_IA.md): los 4 dominios, la integración de extremo a extremo, la revisión de seguridad y el frontend React/Vite están implementados y probados. El Sprint 2 se repartió entre Camilo Sepúlveda, quien integró los dominios y revisó la seguridad, y Logan Silva, quien construyó el frontend sobre las rutas existentes.

5. Arquitectura y persistencia de datos

5.1 Arquitectura

El sistema se divide en backend (Python 3 + FastAPI, carpeta backend/app/) y frontend (React + Vite, carpeta frontend/), comunicados exclusivamente por una API REST/JSON — el frontend no accede a la base de datos directamente (RNF-05). Se eligió FastAPI sobre Flask porque valida automáticamente cada entrada con Pydantic (relevante para FR-01, FR-05, FR-12), y genera documentación interactiva (/docs) útil para probar la API antes de tener frontend.

Para la entrega se compila el frontend (npm run build) directamente a backend/static/, y FastAPI sirve ese build como archivos estáticos desde el mismo puerto que la API (RNF-06): quien evalúa levanta el sistema completo con un solo comando (uvicorn app.main:app), sin instalar Node ni configurar CORS. Durante el desarrollo, el equipo sigue usando dos procesos separados (uvicorn + npm run dev con proxy) para mantener la recarga en caliente de React. El frontend (frontend/src/) usa react-router-dom para las rutas (catálogo público, detalle de paquete, registro/login de cliente, historial de reservas, y login/CRUD de administrador) y un contexto de autenticación (auth.jsx) que guarda el token y el rol en localStorage, igual que hace el backend: el rol que decide qué puede hacer un usuario sale siempre del token, nunca de lo que el frontend decide mostrar.

5.2 Persistencia de datos

La persistencia usa sqlite3 de la librería estándar de Python (sin ORM). El esquema está en backend/app/database.py y se crea al iniciar la API. Cada tabla corresponde a una clase del modelo de dominio (Figura 5), salvo paquete_destinos, que materializa la agregación Paquete–Destino. Las restricciones de columna expresan las reglas que dependen solo de la forma de una fila:

Tabla	Reglas de negocio expresadas en el esquema
destinos	R1 (nombre UNIQUE sin distinguir mayúsculas), R2 (costo_base > 0), duración > 0, R8 (columna disponible)
paquetes	R5 (cupos_maximo > 0; fecha_regreso > fecha_salida), R6 (margen ≥ 0), R7 (precio_por_persona), S2 (estado BORRADOR o PUBLICADO)

Tabla	Reglas de negocio expresadas en el esquema
paquete_destinos	R3 y R4: relación N:N; la clave primaria (paquete_id, destino_id) impide repetir un destino en un paquete, y la clave foránea impide borrar un destino usado en algún paquete
clientes	R9 (correo UNIQUE), R10 (hash_contraseña: nunca la contraseña en texto plano)
administradores	S1 (correo UNIQUE), R10 (hash_contraseña). No hay registro público: se crean desde la consola con python -m app.crear_administrador
reservas	R12 (cliente, paquete y fecha de emisión), R13 (total), R16 (cantidad_personas ≥ 1)

Las reglas que dependen de otros datos o de la fecha del día —R3 (2 a 5 destinos), R8 (eliminar o marcar no disponible según el destino esté en algún paquete), R14 (cupó disponible) y R15 (fecha de salida vigente)— se validan en las clases de dominio (backend/app/dominio/), no en el esquema ni en las rutas de la API. El esquema actúa como segunda línea de defensa: en una prueba en que se desactivó a propósito la regla R8 en la API, la clave foránea de SQLite igualmente impidió borrar el destino. Todas las consultas usan parámetros (?) y nunca concatenan datos de entrada (sección 7.3).

5.3 CRUD por dominio

Avance al 1 de octubre: el Sprint 1 se completó antes de lo planificado (los 4 dominios, HU-01 a HU-04) y, como parte del Sprint 2, se integraron en un flujo único y se revisó la seguridad del sistema integrado (sección 7.3). Las rutas que modifican el catálogo exigen sesión de administrador (S1, RNF-03): sin token responden 401 y con un token de cliente, 403. Rutas disponibles (documentación interactiva en /docs):

Método	Ruta	Operación	Acceso	Req.
GET	/api/destinos	Listar el catálogo (filtro opcional solo_disponibles)	Admin	FR-04
GET	/api/destinos/{id}	Consultar un destino	Admin	FR-04
POST	/api/destinos	Registrar un destino	Admin	FR-01
PUT	/api/destinos/{id}	Modificar un destino	Admin	FR-02
DELETE	/api/destinos/{id}	Dar de baja: elimina o marca no disponible	Admin	FR-03
GET	/api/paquetes	Listar todos los paquetes, incluidos los borradores	Admin	FR-05 a FR-07
GET	/api/paquetes/{id}	Detalle de cualquier paquete	Admin	FR-06, FR-08
POST	/api/paquetes	Crear un paquete en borrador con precio calculado	Admin	FR-05, FR-06
PUT	/api/paquetes/{id}	Modificar un paquete en borrador	Admin	FR-05
POST	/api/paquetes/{id}/publicar	Publicar y fijar el precio	Admin	FR-07
DELETE	/api/paquetes/{id}	Eliminar un paquete en borrador	Admin	S3
GET	/api/paquetes/publicados	Paquetes publicados y vigentes, con precio y cupo disponible	Público	FR-08
GET	/api/paquetes/publicados/{id}	Detalle de un paquete publicado (un borrador responde 404)	Público	FR-08, S2
POST	/api/clientes	Registrar un cliente	Público	FR-09
POST	/api/clientes/sesiones	Iniciar sesión de cliente; devuelve un token	Público	FR-10
GET	/api/clientes/yo	Perfil propio (único lugar con RUT y teléfono)	Cliente	FR-11, R17

Método	Ruta	Operación	Acceso	Req.
POST	/api/reservas	Reservar un paquete publicado y vigente, con cupo	Cliente	FR-12 a FR-14
GET	/api/reservas	Historial propio de reservas	Cliente	FR-15
POST	/api/administradores/sesiones	Iniciar sesión de administrador; devuelve un token	Público	S1, FR-10
GET	/api/administradores/yo	Perfil del administrador	Admin	S1

Calidad: 140 pruebas automatizadas (pytest), unitarias del dominio y de la API sobre SQLite real. Además de cubrir cada regla de R1 a R17 y casos borde (redondeo del precio, margen NaN, nombres con distinta capitalización, intento de inyección SQL, token firmado con otra clave, contraseña de más de 72 bytes), incluyen una prueba de integración que recorre el caso completo con los 4 dominios (test_integracion.py), una prueba de 20 reservas simultáneas sobre un cupo de 5 (test_concurrencia.py) y las pruebas de autorización por rol. Para comprobar que las pruebas detectan errores reales se desactivaron a propósito R7, R8 y cada una de las correcciones de seguridad de la sección 7.3; en todos los casos fallaron las pruebas correspondientes.

6. Justificación técnica y uso crítico de herramientas de IA

Todo uso de IA durante el desarrollo se registra en VALIDACION_IA.md con la misma estructura: objetivo, implementación, revisión técnica (qué propuso la IA o qué alternativa se evaluó, y por qué se adoptó, modificó o descartó) y validación. A continuación, se resumen las decisiones más relevantes tomadas hasta ahora, como evidencia de evaluación crítica y no de copia directa de sugerencias:

Decisión	Alternativas evaluadas	Criterio de adopción
Framework de backend	Flask vs. FastAPI	Se adoptó FastAPI por la validación automática de tipos con Pydantic y la documentación interactiva (/docs).
Servidor de entrega	Dos procesos separados vs. servidor único	Se adoptó servidor único (FastAPI sirve el build de React) para que la evaluación no dependa de instalar Node ni configurar CORS.
Frontend	Create React App vs. Vite	Se adoptó Vite por ser el estándar vigente (CRA está deprecado) y por su configuración más simple de proxy/build para el patrón de servidor único.
Alcance de la validación en el esquema	Expresar R3/R4/R14/R15 como restricciones SQL vs. en el dominio	Se descartaron como restricciones de esquema porque dependen de conteos y fechas en tiempo de ejecución, no de la forma de una fila; se documentó explícitamente para no dejarlo implícito.
Ubicación de las reglas de negocio	En las rutas de FastAPI, en los esquemas Pydantic o en las clases de dominio	Se adoptó el dominio como única fuente de verdad, fiel al diagrama de clases y a RNF-05. Pydantic valida solo tipos y largos, para que una regla no quede duplicada en dos capas que puedan contradecirse.
Margen y montos	float directo vs. Decimal con redondeo explícito	Se adoptó Decimal con redondeo a peso entero (S6): con float, $83.000 \times 1,15$ da 95.449,999... y al truncar se cobraría un peso menos (95.449 en vez de 95.450). Además, se rechaza un margen NaN o infinito, que la comparación «margen ≥ 0 » no detecta.
Ciente HTTP de pruebas	httpx vs. httpx2	La versión instalada de Starlette marca httpx como obsoleto para TestClient; se cambió a httpx2 y se verificó que las 49 pruebas siguen pasando sin advertencias.
Hash de contraseña	passlib[bcrypt] vs. bcrypt directo	Se intentó passlib[bcrypt] (sugerido en la sección 7.1 original) y falló con AttributeError: module 'bcrypt' has no attribute '__about__' — passlib 1.7.4 (sin actualizaciones desde 2020) no es compatible con bcrypt ≥ 4.1 . Se cambió a la librería bcrypt directamente, manteniendo el mismo algoritmo y factor de costo.
Clave JWT (JWT_SECRET)	Clave por defecto en desarrollo vs. obligatoria al arrancar	Una clave por defecto permitiría fabricar tokens de administrador si alguien la conoce, así que no hay valor de respaldo. Se decidió que la aplicación no arranque sin una clave de al menos 32 bytes, con un mensaje claro, en vez de fallar en el primer login con un 500 (Cambio 15).

Decisión	Alternativas evaluadas	Criterio de adopción
Autorización del catálogo	Dejarlo abierto (ningún FR lo exige, Cambio 7) vs. rol de administrador con la herencia del diagrama de clases	Se adoptó el rol de administrador: el supuesto S1, RNF-03 y el diagrama de clases (Usuario → Cliente / Administrador, puede_gestionar_catálogo()) ya lo modelaban, y sin él cualquier persona podía borrar el catálogo sin iniciar sesión. Se implementó con herencia y polimorfismo, como en la Figura 5.
Método de revisión de seguridad	Solo análisis estático (SonarCloud) vs. ataques medidos contra el servidor real más Bandit, pip-audit y SonarLint	El análisis estático no detectó las dos fallas más graves (sobreventa por concurrencia y enumeración de correos por tiempo de respuesta), que solo aparecieron al atacar el servidor en ejecución. Cada hallazgo se demostró con una medición antes de corregirlo y se volvió a medir después (sección 7.3).
Servir el frontend compilado	StaticFiles(html=True) solo vs. comodín de ruta + index.html	StaticFiles(html=True) sirve index.html en "/" pero responde 404 ante una ruta propia de React Router (ej. /admin, /paquetes/3) porque no existe un archivo con ese nombre; se reprodujo navegando directo a esas URLs sobre el build servido. Se reemplazó por una ruta comodín ("/{ruta_spa:path}") que sirve el archivo estático si existe y si no, index.html, dejando que react-router resuelva la ruta en el navegador.

Este mismo criterio — nunca incorporar una sugerencia de IA sin verificarla contra las reglas de negocio del caso — se mantiene para el código de cada dominio que se desarrolle en el Sprint 1, registrando cada cambio en VALIDACION_IA.md antes de integrarlo.

7. Seguridad

Esta sección describe los mecanismos de seguridad implementados en los Sprints 1 y 2 —diseñados antes de escribir código, por instrucción del profesor— con precisión de librería, algoritmo y parámetro, y el resultado de la revisión de seguridad del sistema integrado (7.3).

7.1 Autenticación

Mecanismo	Decisión de diseño
Hash de contraseña	bcrypt (PyPI oficial), factor de costo 12, usado directamente sin passlib: su última versión (1.7.4) es incompatible con bcrypt ≥ 4.1 (ver sección 6). Se descarta cualquier hash reversible o sin salt (MD5, SHA-256 plano).
Token de sesión	PyJWT (PyPI oficial), algoritmo HS256 fijado al decodificar (se rechaza un token sin firma, con algoritmo «none»), expiración de 30 minutos. La clave secreta se lee de la variable de entorno JWT_SECRET, nunca escrita en el código, y debe tener al menos 32 bytes (256 bits); si falta o es más corta, la aplicación no arranca y lo indica (sección 6).
Rol en el token	El token lleva el rol (cliente o administrador) y se rechaza si no lo trae. Sin el rol, el token del cliente 1 serviría también como el del administrador 1, porque se guardan en tablas distintas con identificadores que pueden coincidir.
Validación de credenciales	Ante un correo inexistente o una contraseña incorrecta la API responde el mismo mensaje genérico y tarda lo mismo: si el correo no existe, igual se ejecuta una verificación bcrypt contra un hash ficticio. Sin esto, la diferencia de tiempo (2 ms contra 214 ms) revelaba qué correos están registrados (sección 7.3).
Alcance del token	Reservar y consultar reservas exige un token de cliente; el cliente_id sale del token, nunca de un parámetro (R11). Las rutas que modifican el catálogo exigen un usuario que pueda gestionarlo: se le pregunta puede_gestionar_catologo() (polimorfismo) en vez de revisar su tipo (S1, RNF-03).
Cuentas de administrador	No hay registro público de administradores: se crean desde la consola (python -m app.crear_administrador), que pide la contraseña mostrando un asterisco por carácter, exige al menos 12 caracteres y nunca la recibe por la línea de comandos, donde quedaría en el historial.

7.2 Protección de datos sensibles

El RUT y el teléfono de un cliente son datos sensibles según R17. El diseño los protege en cuatro puntos concretos, no solo como intención general:

- Los modelos de respuesta (Pydantic) de Cliente excluyen rut y telefono en cualquier listado administrativo; solo el propio cliente autenticado los ve al consultar su perfil.
- Los mensajes de error de validación (ej. RUT con formato inválido) nunca repiten el valor recibido cuando el campo es rut o telefono, para no filtrarlo en un log de errores del servidor.
- La contraseña no se registra en ningún log, ni siquiera ya hasheada; solo se calcula en memoria para comparar y se descarta.
- El RUT se valida con su dígito verificador (módulo 11) y se guarda normalizado (12345678-5); el mensaje de error no repite el RUT recibido.

7.3 Evaluación de seguridad asistida por IA

La revisión se hizo el 1 de octubre, con los 4 dominios ya integrados, combinando revisión manual, ataques contra el servidor en ejecución, Bandit, pip-audit y SonarCloud. SonarCloud analizó el commit

5c495d17 con Quality Gate aprobado y 0 issues en New Code; Bandit cubre las reglas de inyección SQL, secretos y manejo de excepciones de la checklist. Hallazgos y correcciones:

Hallazgo	Severidad	Evidencia (antes)	Corrección	Verificación (después)
Sobreventa por reservas simultáneas (R14)	Alta	20 reservas simultáneas de 1 persona sobre un paquete con cupo 5: se aceptaron las 20 (cupo disponible –15). Leer el cupo e insertar la reserva eran pasos separados.	Leer el cupo y registrar la reserva en una sola transacción exclusiva (BEGIN IMMEDIATE).	5 aceptadas y 15 rechazadas (test_concurrencia.py).
Catálogo modificable sin sesión (S1, RNF-03)	Alta	POST /api/destinos sin token respondía 201: cualquier persona podía crear, modificar o borrar destinos y paquetes.	Rol de administrador con la herencia Usuario → Cliente / Administrador del diagrama de clases.	Sin token: 401; con token de cliente: 403; 10 rutas probadas.
Token sin rol	Alta (latente)	Al agregar administradores, un token con solo el id habría permitido que el cliente 1 actuara como el administrador 1.	Rol obligatorio en el token; se rechaza uno sin rol o con un rol desconocido.	Pruebas de token sin rol, con rol inventado y de cliente usado como administrador.
Enumeración de correos por tiempo de respuesta	Media	Inicio de sesión fallido: 214 ms si el correo existe y 2 ms si no (mediana de 8 intentos).	Verificar contra un hash bcrypt ficticio cuando el correo no existe.	216 ms contra 220 ms; prueba que cuenta las verificaciones bcrypt.
Clave JWT débil aceptada	Baja	La API aceptaba cualquier JWT_SECRET, incluso de un carácter.	Mínimo de 32 bytes.	Prueba con una clave corta.
RUT sin validar	Baja	Se aceptaba cualquier texto como RUT (por ejemplo, «abc»).	Dígito verificador (módulo 11) y normalización; el error no repite el RUT.	12 casos de prueba.
Contraseña literal en el código (SonarLint S6437)	Falso positivo	SonarLint marcó el texto usado para generar el hash ficticio del hallazgo de enumeración.	Se reemplazó por bytes aleatorios (módulo secrets), lo que además impide adivinar el hash ficticio.	SonarLint sin avisos.

Checklist de la revisión:

- **Inyección SQL:** todas las consultas usan parámetros (?). Bandit (regla B608) no encontró consultas armadas con texto en 1.270 líneas; como control, sí la detectó en un archivo de prueba con concatenación. Una prueba guarda un nombre con «'; DROP TABLE» y lo recupera literal.
- **Secretos en el código:** ninguno; la clave JWT solo se lee de JWT_SECRET (Bandit sin hallazgos B105 ni B106).
- **Manejo de excepciones:** un error no controlado responde «Error interno del servidor.» sin traza, ruta de archivo ni SQL (test_errores.py), y los errores de validación no repiten el valor recibido.
- **Dependencias:** pip-audit no encontró vulnerabilidades conocidas en las versiones fijadas en requirements.txt.

Riesgos aceptados dentro del alcance: (a) el registro de clientes informa «Ya existe un cliente registrado con ese correo», porque R9 exige que el correo sea único y el cliente necesita saberlo; esto permite comprobar si un correo está registrado, pero solo desde el registro, no desde el inicio de sesión. (b) No

hay límite de intentos de inicio de sesión: con bcrypt de costo 12 cada intento tarda unos 200 ms, lo que encarece un ataque de fuerza bruta pero no lo impide; un límite por dirección IP requiere infraestructura fuera del alcance de la primera versión.

8. Conclusiones

El sistema cumple los requerimientos funcionales (FR-01 a FR-17) y no funcionales (RNF-01 a RNF-06) de la sección 2. La trazabilidad de la sección 3 se mantuvo durante la implementación: cada endpoint y cada regla de negocio se puede rastrear hasta un requerimiento y hasta una prueba automatizada (140 pruebas al cierre).

El aprendizaje técnico más importante fue medir el sistema en ejecución. El análisis estático (Bandit, pip-audit y SonarCloud) no detectó dos fallas graves: la sobreventa de cupo por concurrencia (20 reservas aceptadas con cupo 5) y la enumeración de correos por tiempo de respuesta (214 ms frente a 2 ms). Solo aparecieron al atacar el servidor en ejecución, y se corrigieron con mediciones antes y después.

El uso de herramientas de IA aceleró el desarrollo, pero cada propuesta se verificó contra las reglas del caso y contra pruebas que fallan si se rompe la regla. Eso dejó decisiones documentadas: el descarte de passlib (incompatible con bcrypt 4.1 o superior), la corrección de una validación de correo que aceptaba dos arrobas tras una revisión posterior y la incorporación de un arranque que exige la clave JWT.

Limitaciones conocidas

- La persistencia usa SQLite. En el plan gratuito de Render el disco es efímero: la demo se reinicia cuando el servicio se reinicia.
- No hay límite de intentos de inicio de sesión; es un riesgo abierto.
- La cuenta de administrador de la demo se crea desde variables de entorno. Es aceptable para una demostración, no para producción.

Trabajo futuro

- Límite de intentos y bloqueo temporal en el inicio de sesión.
- Base de datos de servidor (PostgreSQL) con respaldos.
- El nombre del paquete en el historial del cliente (hoy solo se muestra su número).
- Control de versiones para la edición concurrente del catálogo.

En síntesis, el sistema es funcional, está probado y desplegado, y deja documentados sus límites y sus decisiones.

Enlaces del proyecto

Repositorio del código: https://github.com/digicamilo-lgtm/viajes_ventura-CS.LS

Demostración desplegada: <https://viajes-aventura-o5p1.onrender.com>

Referencias bibliográficas

Cuevas Álvarez, A. (2016). Python 3. RA-MA Editorial.

Jiménez de Parga, C. (2021). UML: arquitectura de aplicaciones en Java, C++ y Python (1.ª ed.). Ra-Ma.

Sánchez Palacio, A. (2025). ChatGPT y OpenAI: desarrollo y uso de herramientas de inteligencia artificial generativa. RA-MA Editorial.